# សប្តាហ៍ទី 5 · ការពិនិត្យ input

## Programming for Information Systems

កុំទុកចិត្ត input: ពិនិត្យនៅមាត់ទ្វារ

# ចាប់ផ្តើមថ្នាក់ · ចែករំលែកកិច្ចការស្រាវជ្រាវទី 3

ក្នុងក្រុមបីនាក់ ចែករំលែកការពិនិត្យ input ដែលអ្នកបានស្រាវជ្រាវ។

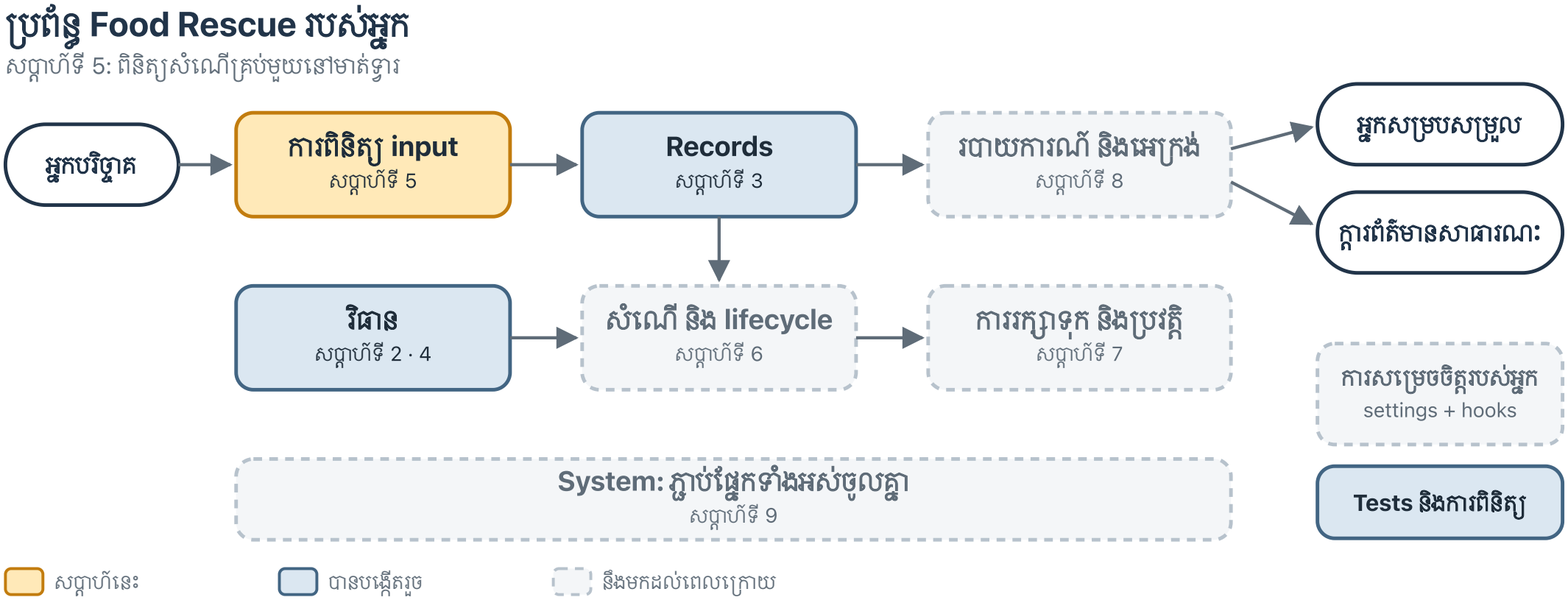
1. តើអ្នកបានសិក្សា form ណា? តើវាសុំឱ្យមនុស្សវាយអ្វី?
2. តើអ្នករកឃើញការពិនិត្យអ្វី? តើមនុស្សឃើញ **សារ** អ្វី? តើវាប្រើពាក្យសាមញ្ញឬទេ?
3. ប្រាប់ input មួយដែលវា **ទទួលយក តែគួរបដិសេធ** ឬ **បដិសេធ តែគួរទទួលយក**។ តើនរណាទទួលផលប៉ះពាល់?

> 📝 សរសេរគំនិតមួយពីមិត្តរួមថ្នាក់ ដែលអ្នកចង់ចងចាំ។

# សប្តាហ៍នេះ

| | |
|---|---|
| **សំណួរ** | តើប្រព័ន្ធគួរធ្វើអ្វី ពេល input ខុស? |
| **Python** | `strip`, `lower`, `int()`, `try`/`except ValueError`, list នៃ error, return value ពីរ |
| **គោលការណ៍ទី 5** | កុំទុកចិត្ត input |
| **អ្នកបង្កើត** | `clean_offer(raw)` → `(offer, errors)` |

# ប្រព័ន្ធរបស់អ្នករហូតមកដល់ពេលនេះ



# គំនិតសំខាន់: ពិនិត្យនៅមាត់ទ្វារ

- នៅព្រលានយន្តហោះ បុគ្គលិកពិនិត្យលិខិតឆ្លងដែនរបស់អ្នក **មុនពេល** អ្នកឡើងយន្តហោះ។
- ប្រព័ន្ធក៏មានទ្វារដែរ: form មួយ file មួយ សារមួយ។
- មាត់ទ្វារគឺជាកន្លែង **ចំណាយពិចបំផុត** ក្នុងការបញ្ឈប់ទិន្នន័យមិនល្អ។
- ពេលទិន្នន័យមិនល្អចូលដល់ខាងក្នុងហើយ វារាលដាលទៅក្នុងផលប៉ុក របាយការណ៍ និងការសម្រេចចិត្ត។

> មនុស្សមិនមែនធ្វេសប្រហែសដោយចេតនាទេ។ ប្រព័ន្ធល្អ **រំពឹងទុក** ថានឹងមាន input ខុស។

# value គ្រប់មួយពី file គឺជា text

```
donor,contact,food,category,portions,hours_left
Sunrise Bakery,sunrise@example.org,Bread rolls,bread,30,5
Student Club Bake Sale,club@example.org,Cupcakes,snacks,many,6
Hotel Kitchen,hotel@example.org,Steamed rice,rice meals,1200,8
,,Mango slices,fruit,10,6
```

```
rows = fr.validation.read_offers_csv("food_rescue_kit/data/offers.csv")
print(rows[0]["portions"], type(rows[0]["portions"]))   # 30 <class 'str'>
```

- file CSV មានតែតួអក្សរប៉ុណ្ណោះ។ វាមិនអាចប្រាប់ថា "នេះជាលេខ" បានទេ។

# រៀបចំ text ឱ្យស្អាត: `strip()` និង `lower()`

```
typed = "  Dara "
print(typed == "Dara")                  # False
print(typed.strip() == "Dara")          # True
print(" Bread ".strip().lower())        # bread
print(" Bread " in CATEGORIES)          # False
print(" Bread ".strip().lower() in CATEGORIES)   # True
```

- **method** គឺជា function ដែលជារបស់ value មួយ: `text.strip()`។
- វា return text ថ្មី។ text ដើមមិនប្រែប្រួលទេ។

# `int()`: text ទៅជាចំនួនគត់ ឬ error

```
print(int("30") + 4)      # 34
print(int(" 12 "))        # 12   (ដកឃ្លានៅចុងសងខាង មិនអីទេ)
print(int("-3"))          # -3   ជាចំនួនគត់ ប៉ុន្តែសមហេតុផលទេ?
print(int("many"))        # ValueError
```

```
ValueError: invalid literal for int() with base 10: 'many'
```

- ការបម្លែងគឺជាការពិនិត្យមួយ។ **range** គឺជាការពិនិត្យទីពីរ។

# `try` / `except ValueError`

```
quantity_text = "two"
try:
    quantity = int(quantity_text)
    print("Order for", quantity, "phones")
except ValueError:
    print("Please type the quantity as a number, for example 2.")
```

- ទុក `try` ឱ្យ **តូច**: នៅជុំវិញតែការបម្លែងប៉ុណ្ណោះ។
- ចាប់ **តែ** `ValueError` ដែលជាបញ្ហាដែលអ្នករំពឹងទុក។

# ប្រមូលបញ្ហាទាំងអស់

```
errors = []
if name.strip() == "":
    errors.append("name is required.")
try:
    age = int(age_text)
except ValueError:
    errors.append("age must be a whole number, for example 19.")
```

- ពិនិត្យ **គ្រប់យ៉ាង** ដើម្បីឱ្យមនុស្សអាចកែបញ្ហាទាំងអស់ក្នុងពេលតែមួយ។
- ពាក្យសាមញ្ញ: **អ្វី** ដែលខុស និង **របៀប** កែវា។

# return value ពីរ ហើយ unpack វា

```
def check_age(age_text):
    try:
        age = int(age_text)
    except ValueError:
        return None, ["age must be a whole number, for example 19."]
    if age < 16:
        return None, ["volunteers must be 16 or older."]
    return age, []

age, errors = check_age("17")     # 17 []
```

# គោលការណ៍ទី 5 · កុំទុកចិត្ត input

1. **ពិនិត្យនៅមាត់ទ្វារ** មុនពេលរក្សាទុកអ្វីមួយ។
2. **ពន្យល់** បញ្ហានីមួយៗជាពាក្យសាមញ្ញ។
3. **កុំប្តូរអ្វីទាំងអស់** ពេល input មិនល្អ។

> ការទុកចិត្តមិនមែននិយាយពីមនុស្សទេ។ មនុស្សស្មោះត្រង់ ក៏នៅតែវាយ "many" ដែរ។

# ⚖️ ល្អ ឬមានហានិភ័យ?

**Version A** (ពីជំនួយការ AI)

```
def add_offer(raw):
    offer = {
        "donor": raw["donor"],
        "category": raw["category"],
        "portions": int(raw["portions"]),
    }
    saved_offers.append(offer)
    return "OK"
```

វា crash នៅ "many"។ អ្នកបានប្រាប់វា។

**Version B** ("កែរួចហើយ! វានឹងមិន crash ទៀតទេ។")

```
def add_offer(raw):
    offer = {"donor": raw["donor"]}
    saved_offers.append(offer)  # រក្សាទុកមុន
    try:
        offer["portions"] = int(raw["portions"])
    except:
        offer["portions"] = 0   # មិន crash
    if raw["category"] not in CATEGORIES:
        return "error"
    offer["category"] = raw["category"]
    return "OK"
```

# អ្វីដែល Version B បានធ្វើចំពោះថ្ងៃគំរូ

| ជួរ | អ្វីដែល B បានរក្សាទុក | នរណារងគ្រោះ |
|---|---|---|
| Cupcakes, "many" | 0 ចំណែក | Student Club គិតថាវាត្រូវបានទទួលយក; គ្មាននរណាទៅយកទេ |
| Noodle soup, "soup" | សំណើពាក់កណ្តាល គ្មានប្រភេទ | អ្នកសម្របសម្រួលទទួលបាន record ខូច |
| Steamed rice, 1200 | ទទួលយក | ក្រុមមួយទៅយកអាហារ ដែលគ្មានអង្គការដៃគូណាអាចទទួលបាន |
| Mango slices, គ្មានអ្នកបរិច្ចាគ | ទទួលយក | គ្មាននរណាដឹងថាត្រូវទៅយកនៅឯណាទេ |

**ខុសដោយស្ងាត់ៗ អាក្រក់ជាងការបដិសេធយ៉ាងច្បាស់។** crash ត្រូវបានគេកត់សម្គាល់ ប៉ុន្តែលេខ 0 ស្ងាត់ៗ ចូលទៅក្នុងផលប្លូក។

# អ្នកបង្កើត: `clean_offer(raw)`

| Input | Return |
|---|---|
| ជួរល្អ | `(offer, [])`: text ត្រូវបាន strip, category ជាអក្សរតូច, លេខជា `int` |
| ជួរមិនល្អ | `(None, [every problem in plain words])`: បញ្ហាទាំងអស់ ជាពាក្យសាមញ្ញ |

- **អ្នកបរិច្ចាគ** អានសារ។ **អ្នកសម្របសម្រួល** ទទួលបាន record ស្អាត។
- ដែនកំណត់គឺជា constant: `MIN_PORTIONS`, `MAX_PORTIONS`, `REJECT_IF_HOURS_LEFT_BELOW`, `CATEGORIES`, `REQUIRED_FIELDS`។

# TODO ទាំងបួន

```
def to_whole_number(text):
    ...                          # TODO 1: try int(text), except ValueError → None

def clean_offer(raw):
    donor = raw.get("donor", "")          # TODO 2: .strip() (និង .lower())
    ...                                   # field ដែលត្រូវតែបំពេញ: សរសេរបួច
    if portions is None:
        errors.append("portions must be a whole number, for example 12.")
    # TODO 3: elif portions នៅក្រៅ MIN_PORTIONS..MAX_PORTIONS
    # TODO 4: category ត្រូវតែនៅក្នុង CATEGORIES
```

- ភាគច្រើននៃ function ត្រូវបានសរសេរបួចសម្រាប់អ្នក។ អ្នកបំពេញតែបន្ទាត់សំខាន់ៗ។
- សាកល្បងវាលើជួរញ៉ែញ៉ែមួយ និងជួរមិនល្អមួយ **មុនពេល** អ្នក run ការពិនិត្យ។

# ពិនិត្យរា រួចភ្ជាប់វាចូលប្រព័ន្ធ

```
fr.check("clean_offer", clean_offer, MY_SETTINGS)

my_system = fr.System(MY_SETTINGS, hooks={"clean_offer": clean_offer})
my_system.run_day()          # 🚫 = ជួរដែលទ្វាររបស់អ្នកបានបដិសេធ
```

- ទ្វារដែលបដិសេធអ្វីៗទាំងអស់ ជាប់ 6 ក្នុងចំណោម 7 ករណី។
- នោះហើយជាមូលហេតុដែលករណី **ទីមួយ** គឺជាជួរល្អដែលមានដកឃ្លាលើស។
- ការបដិសេធត្រូវតែមិនប្តូរអ្វីទាំងអស់: `records()` មុន == ក្រោយ។

# 🧭 ការសម្រេចចិត្តរបស់អ្នក · តើទ្វារទទួលយកអ្វីខ្លះ?

```
MIN_PORTIONS = 1
MAX_PORTIONS = 200
REJECT_IF_HOURS_LEFT_BELOW = 1
REQUIRED_FIELDS = ["donor", "food", "category", "portions", "hours_left"]
```

- **ចំណែក:** តើ 1 ចំណែក សមនឹងការធ្វើដំណើរទេ? តើមានអង្គការដែគួណាអាចទទួល 200 បានទេ?
- **ម៉ោងនៅសល់:** ពេលនៅសល់ 1 ម៉ោង តើអ្នកស្ម័គ្រចិត្តនៅទាន់ទៅដល់ទេ?
- **field ដែលត្រូវតែបំពេញ:** តើ `contact` គួរតែជា field ដែលត្រូវតែបំពេញឬទេ?
  - **គាំទ្រ:** ក្រុមការងារអាចរៀបចំការទៅយក ហើយទូរស័ព្ទប្រាប់ពេលមានបញ្ហា។
  - **ប្រឆាំង:** ប្រព័ន្ធរក្សាទិន្នន័យផ្ទាល់ខ្លួនច្រើនជាងមុន ហើយត្រូវតែការពារ។

> កំណត់ត្រាការសម្រេចចិត្ត: ជម្រើសរបស់ខ្ញុំ · ជម្រើសផ្សេង · មូលហេតុ · នរណាទទួលផលប៉ះពាល់

# កិច្ចការសប្តាហ៍នេះ

- ⭐ **មូលដ្ឋាន:** P1–P4 ការពិភាក្សា ⚖️ `clean_offer` ដែលមាន ✅ ទាំងអស់ ភ្ជាប់ចូលប្រព័ន្ធ កំណត់ត្រាការសម្រេចចិត្ត។
- ⭐⭐ **បន្ថែម:** លេខដែលគួរឱ្យភ្ញាក់ផ្អើល (`int("១២")` គឺ 12!) និងសារជ័រសរាយតែមួយសម្រាប់អ្នកបរិច្ចាគ។
- 🔎 **កិច្ចការស្រាវជ្រាវទី 4 (5%):** របៀបដែលប្រព័ន្ធពិតរក្សាទិន្នន័យ និងប្រវត្តិ: រក្សាទុកអ្វី ប្រើ ID អ្វី មានអ្វីកើតឡើងពេលកែតម្រូវ នរណាត្រូវការប្រវត្តិ។ ចែករំលែកវានៅពេលចាប់ផ្តើមថ្នាក់សប្តាហ៍ទី 6។

# សប្តាហ៍ក្រោយ · Objects និង lifecycle

សំណើដែលស្នាតហើយ នៅតែមានដំណើរបស់វា: **ថ្មី** → **បានអនុម័ត** → **បានទៅយក** → **បានប្រគល់**។ អ្នកនឹងធ្វើឱ្យប្រាកដថា វាអាចប្តូរបានតែតាមផ្លូវត្រឹមត្រូវប៉ុណ្ណោះ។